

# ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №31

## Тема: Створення алгоритмів з геометрією

**Дисципліна:** Алгоритмізація і методи обчислень

**Спеціальність:** Професійна освіта (Цифрові технології)

**Тривалість:** 80 хвилин

### Мета роботи

Опанувати основні алгоритми обчислювальної геометрії на площині: відстань і кут, векторний і скалярний добутки, орієнтація трьох точок, площа багатокутника, перетин відрізків, належність точки багатокутнику, побудова опуклої оболонки; навчитися працювати з точністю обчислень у геометричних задачах.

### Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), бібліотека matplotlib (для графіків).

---

### Короткі теоретичні відомості

**1. Точки та вектори.** Точку на площині задають парю координат  $(x, y)$ . Вектор  $AB = (x_B - x_A, y_B - y_A)$ . Довжина вектора  $|v| = \sqrt{v_x^2 + v_y^2}$ ; відстань між точками – довжина вектора, що їх з'єднує.

**2. Скалярний і векторний (косий) добутки.** Для векторів  $u = (u_x, u_y)$ ,  $v = (v_x, v_y)$ :

- **скалярний добуток**  $u \cdot v = u_x v_x + u_y v_y = |u| \cdot |v| \cdot \cos \varphi$ ; дорівнює 0 для перпендикулярних векторів; за ним знаходять кут:  $\varphi = \arccos(u \cdot v / (|u| |v|))$ ;
- **косий добуток**  $u \times v = u_x v_y - u_y v_x = |u| \cdot |v| \cdot \sin \varphi$  (зі знаком); його модуль дорівнює площі паралелограма на цих векторах; дорівнює 0 для колінеарних векторів.

**3. Орієнтація трьох точок.** Знак виразу  $\text{cross}(A, B, C) = (B - A) \times (C - A)$  показує поворот ламаної  $A \rightarrow B \rightarrow C$ :  $> 0$  — проти годинникової стрілки (ліворуч),  $< 0$  — за годинниковою стрілкою (праворуч),  $= 0$  — точки лежать на одній прямій. Ця операція — основа майже всіх алгоритмів обчислювальної геометрії, оскільки використовує лише множення й додавання (без ділення та тригонометрії). Для цілих координат вона **точна**.

**4. Площа багатокутника (формула шнурівки, формула Гаусса).** Для вершин  $P_0, P_1, \dots, P_{n-1}$  у порядку обходу:  $S = \frac{1}{2} \cdot |\sum (x_i y_{i+1} - x_{i+1} y_i)|$ , де індекси беруться за модулем  $n$ . Знак суми показує напрямок обходу (додатний — проти годинникової стрілки).

**5. Перетин відрізків AB та CD.** Відрізки перетинаються, якщо кінці кожного лежать по різні боки від прямої, що містить інший відрізок:  $\text{cross}(A, B, C)$  і  $\text{cross}(A, B, D)$  мають різні знаки та  $\text{cross}(C, D, A)$  і  $\text{cross}(C, D, B)$  мають різні знаки. Окремо розглядають колінеарні випадки (перевіряють перекриття проєкцій).

**6. Належність точки багатокутнику (метод променя).** З точки випускають горизонтальний промінь і підраховують кількість перетинів зі сторонами: непарна кількість — точка всередині, парна — зовні. Складність  $O(n)$ .

**7. Опукла оболонка** множини точок — найменший опуклий багатокутник, що містить усі точки (уявіть гумову стрічку, натягнуту на цвяхи). **Алгоритм Ендрю (monotone chain)**: точки сортують за (x, y); нижню оболонку будують проходом зліва направо, вилучаючи з вершини стека останню точку, доки вона не утворює лівий поворот; верхню — проходом справа наліво; результат — об'єднання. Складність  $O(n \log n)$  (основна витрата — сортування).

**8. Точність.** У геометричних алгоритмах із дійсними числами порівняння з нулем слід робити з допуском  $\epsilon$  ( $\text{abs}(x) < 1e-9$ ); цілі координати дозволяють виконувати обчислення точно.

---

## Хід роботи

### Завдання 1. Ручні розрахунки: векторний добуток, площа, орієнтація

Дано трикутник  $A(0, 0)$ ,  $B(6, 0)$ ,  $C(2, 4)$  та чотирикутник  $P(0, 0)$ ,  $Q(5, 0)$ ,  $R(6, 3)$ ,  $S(1, 4)$ .

*Порядок виконання:*

1. Обчисліть довжини сторін трикутника ABC та його периметр.
2. Обчисліть косий добуток  $AB \times AC$  і за ним — площу трикутника ( $S = \frac{1}{2} \cdot |AB \times AC|$ ); перевірте за формулою «половина основи на висоту».
3. Визначте кут при вершині A за скалярним добутком  $AB \cdot AC$  (результат у градусах).
4. Визначте орієнтацію трійок  $(A, B, C)$ ,  $(A, C, B)$  та  $(A, B, D)$ , де  $D(12, 0)$ ; поясніть значення знака.
5. Обчисліть площу чотирикутника PQRS за формулою шнурівки: складіть таблицю «і —  $x_i \cdot y_{i+1}$  —  $x_{i+1} \cdot y_i$  — різниця»; визначте напрямок обходу за знаком суми.

### Завдання 2. Ручний розв'язок: перетин відрізків і належність точки багатокутнику

Перевірити перетин пар відрізків та належність точок багатокутнику методом променя.

*Порядок виконання:*

1. Для відрізків AB,  $A(0, 0)$ ,  $B(4, 4)$ , та CD,  $C(0, 4)$ ,  $D(4, 0)$ : обчисліть чотири орієнтації  $\text{cross}(A, B, C)$ ,  $\text{cross}(A, B, D)$ ,  $\text{cross}(C, D, A)$ ,  $\text{cross}(C, D, B)$  та зробіть висновок про перетин.
2. Для відрізків  $A(0, 0)$ ,  $B(2, 2)$  та  $C(3, 3)$ ,  $D(5, 5)$  (лежать на одній прямій) поясніть, чому всі орієнтації дорівнюють 0 і як перевірити перекриття проєкцій на вісь x.
3. Для відрізків  $A(0, 0)$ ,  $B(4, 0)$  та  $C(2, 1)$ ,  $D(2, 5)$ : визначте, чи перетинаються вони (кінці C і D по одну сторону від прямої AB).
4. Для багатокутника PQRS із завдання 1 визначте методом променя належність точок  $T_1(3, 2)$ ,  $T_2(7, 2)$ ,  $T_3(3, 6)$ : для кожної точки проведіть горизонтальний промінь праворуч і підрахуйте його перетини зі сторонами.
5. Складіть таблицю «точка — кількість перетинів — належність».

### Завдання 3. Програмна реалізація базових геометричних алгоритмів

Реалізувати мовою Python функції для роботи з точками й багатокутниками та перевірити їх на даних ручних розрахунків.

*Порядок виконання:*

1. Напишіть функції  $\text{dist}(a, b)$ ,  $\text{cross}(a, b, c)$  (орієнтація),  $\text{dot}(u, v)$  та  $\text{angle\_at}(a, b, c)$  (кут при вершині b у градусах).
2. Напишіть функцію  $\text{polygon\_area}(pts)$  за формулою шнурівки; поверніть також напрямок обходу (за знаком суми).

3. Напишіть функцію `segments_intersect(a, b, c, d)`, що враховує загальний і колінеарні випадки (допоміжна функція `on_segment`).
4. Напишіть функцію `point_in_polygon(p, poly)` методом променя (враховуйте, що промінь, який проходить через вершину, не повинен рахуватись двічі: порівняння  $y$ -координат виконуйте з умовою  $(y_1 > p_y) \neq (y_2 > p_y)$ ).
5. Перевірте функції на тестах із завдань 1 та 2 і порівняйте з ручними результатами.

*Приклад реалізації мовою Python:*

python

```
import math

def dist(a, b):
    return math.hypot(b[0] - a[0], b[1] - a[1])

def cross(a, b, c):
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def dot(u, v):
    return u[0] * v[0] + u[1] * v[1]

def angle_at(a, b, c):
    u = (a[0] - b[0], a[1] - b[1])
    v = (c[0] - b[0], c[1] - b[1])
    return math.degrees(math.acos(dot(u, v) / (math.hypot(*u) * math.hypot(*v))))

def polygon_area(pts):
    s = 0
    for i in range(len(pts)):
        x1, y1 = pts[i]
        x2, y2 = pts[(i + 1) % len(pts)]
        s += x1 * y2 - x2 * y1
    return abs(s) / 2, ("проти годинникової" if s > 0 else "за годинниковою")

def on_segment(a, b, p):
    return min(a[0], b[0]) <= p[0] <= max(a[0], b[0]) and min(a[1], b[1]) <= p[1] <= max(a[1], b[1])

def segments_intersect(a, b, c, d):
    d1, d2 = cross(a, b, c), cross(a, b, d)
    d3, d4 = cross(c, d, a), cross(c, d, b)
    if ((d1 > 0 and d2 < 0) or (d1 < 0 and d2 > 0)) and ((d3 > 0 and d4 < 0) or (d3 < 0 and d4 > 0)):
        return True
    return ((d1 == 0 and on_segment(a, b, c)) or (d2 == 0 and on_segment(a, b, d)) or
            (d3 == 0 and on_segment(c, d, a)) or (d4 == 0 and on_segment(c, d, b)))

def point_in_polygon(p, poly):
    inside = False
    n = len(poly)
    for i in range(n):
        (x1, y1), (x2, y2) = poly[i], poly[(i + 1) % n]
        if (y1 > p[1]) != (y2 > p[1]):
            x_cross = x1 + (p[1] - y1) * (x2 - x1) / (y2 - y1)
```

```

        if x_cross > p[0]:
            inside = not inside
    return inside

A, B, C = (0, 0), (6, 0), (2, 4)
print("Сторони:", round(dist(A, B), 3), round(dist(B, C), 3), round(dist(C, A), 3))
print("AB x AC =", cross(A, B, C), "; площа трикутника =", cross(A, B, C) / 2)
print(f"Кут при A = {angle_at(B, A, C):.4f} градусів")
print("Орієнтації:", cross(A, B, C), cross(A, C, B), cross(A, B, (12, 0)))

P = [(0, 0), (5, 0), (6, 3), (1, 4)]
print("Площа PQRS:", polygon_area(P))
print("Перетин (0,0)-(4,4) і (0,4)-(4,0):", segments_intersect((0, 0), (4, 4), (0, 4), (4, 0)))
print("Колінеарні (0,0)-(2,2) і (3,3)-(5,5):", segments_intersect((0, 0), (2, 2), (3, 3), (5, 5)))
print("Колінеарні (0,0)-(3,3) і (2,2)-(5,5):", segments_intersect((0, 0), (3, 3), (2, 2), (5, 5)))
print("(0,0)-(4,0) і (2,1)-(2,5):", segments_intersect((0, 0), (4, 0), (2, 1), (2, 5)))
for t in ((3, 2), (7, 2), (3, 6), (2, 1)):
    print(f"Точка {t} всередині PQRS:", point_in_polygon(t, P))

```

### Результат виконання програми:

```

Сторони: 6.0 5.657 4.472
AB x AC = 24 ; площа трикутника = 12.0
Кут при A = 63.4349 градусів
Орієнтації: 24 -24 0
Площа PQRS: (18.0, 'проти годинникової')
Перетин (0,0)-(4,4) і (0,4)-(4,0): True
Колінеарні (0,0)-(2,2) і (3,3)-(5,5): False
Колінеарні (0,0)-(3,3) і (2,2)-(5,5): True
(0,0)-(4,0) і (2,1)-(2,5): False
Точка (3, 2) всередині PQRS: True
Точка (7, 2) всередині PQRS: False
Точка (3, 6) всередині PQRS: False
Точка (2, 1) всередині PQRS: True

```

*Висновок за результатами: результати програми збігаються з ручними розрахунками: площа трикутника ABC дорівнює 12, кут при вершині A — 63.4349°, площа чотирикутника PQRS — 18 (сума в формулі шнурівки 36, обхід проти годинникової стрілки). Відрізки (0,0)–(4,4) і (0,4)–(4,0) перетинаються, колінеарні відрізки перетинаються лише за наявності перекриття, а відрізок (0,0)–(4,0) із (2,1)–(2,5) не перетинається, оскільки обидва кінці другого лежать по одну сторону від прямої. Метод променя класифікує точки (3, 2) як внутрішню, а (7, 2) та (3, 6) як зовнішні. Зверніть увагу: для цілих координат функція cross працює точно, без похибок округлення.*

### Завдання 4. Опукла оболонка (алгоритм Ендрю) та перевірка

Реалізувати побудову опуклої оболонки множини точок, візуалізувати результат і перевірити коректність на випадкових даних.

#### Порядок виконання:

1. Складіть вручну опуклу оболонку точок (0,0), (4,0), (4,4), (0,4), (2,2), (1,3), (3,1), (2,0): виконайте проходи для нижньої й верхньої оболонки, записуючи стан стека та рішення «вилучити / залишити» для кожної точки.
2. Напишіть функцію `convex_hull(points)` за алгоритмом Ендрю (функції `cross` та `polygon_area` беріть із завдання 3): сортування, нижня оболонка, верхня оболонка, об'єднання без дублювання кінцевих точок; вилучайте точки, доки  $\text{cross} \leq 0$ .

3. Згенеруйте 40 випадкових точок із цілими координатами в діапазоні 0...50 (з `random.seed(5)`); побудуйте оболонку та виведіть кількість її вершин і її площу.
4. Перевірте коректність двома властивостями: (а) усі точки лежать усередині або на межі оболонки (для кожного ребра  $\text{cross} \geq 0$ ); (б) обхід здійснюється проти годинникової стрілки, а оболонка не має «вгнутих» вершин.
5. Побудуйте графік: усі точки, ребра оболонки та її вершини іншим кольором.
6. Дослідіть час побудови оболонки для  $n = 10^3, 10^4, 10^5$  точок і підтвердіть оцінку  $O(n \log n)$ .

*Приклад реалізації мовою Python:*

python

```
import random
import time
import matplotlib.pyplot as plt

def convex_hull(points):
    pts = sorted(set(points))
    if len(pts) <= 2:
        return pts
    lower = []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)
    upper = []
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[::-1]

simple = [(0, 0), (4, 0), (4, 4), (0, 4), (2, 2), (1, 3), (3, 1), (2, 0)]
print("Оболонка простого набору:", convex_hull(simple))

random.seed(5)
points = [(random.randint(0, 50), random.randint(0, 50)) for _ in range(40)]
hull = convex_hull(points)
print("Випадкові точки: унікальних", len(set(points)), ", вершин оболонки", len(hull),
      ", площа", polygon_area(hull)[0])

n_h = len(hull)
inside_ok = all(cross(hull[i], hull[(i + 1) % n_h], p) >= 0 for i in range(n_h) for p
in points)
convex_ok = all(cross(hull[i], hull[(i + 1) % n_h], hull[(i + 2) % n_h]) > 0 for i in
range(n_h))
print("Усі точки всередині оболонки:", inside_ok, "; усі повороти ліві:", convex_ok)

print("\n      n    вершин оболонки    час, c")
for n in (10**3, 10**4, 10**5):
    pts = [(random.randint(0, 10**6), random.randint(0, 10**6)) for _ in range(n)]
    t0 = time.perf_counter()
    h = convex_hull(pts)
    t = time.perf_counter() - t0
    print(f"{n:>7}    {len(h):>14}    {t:>7.4f}")

xs, ys = zip(*points)
plt.figure(figsize=(5.5, 5.5))
plt.scatter(xs, ys, s=18, color="#4a6fa5", label="точки")
hx, hy = zip(*hull + [hull[0]])
plt.plot(hx, hy, "-", color="#c0392b", lw=1.6, label="опукла оболонка")
plt.scatter(hx, hy, s=30, color="#c0392b", zorder=3)
```

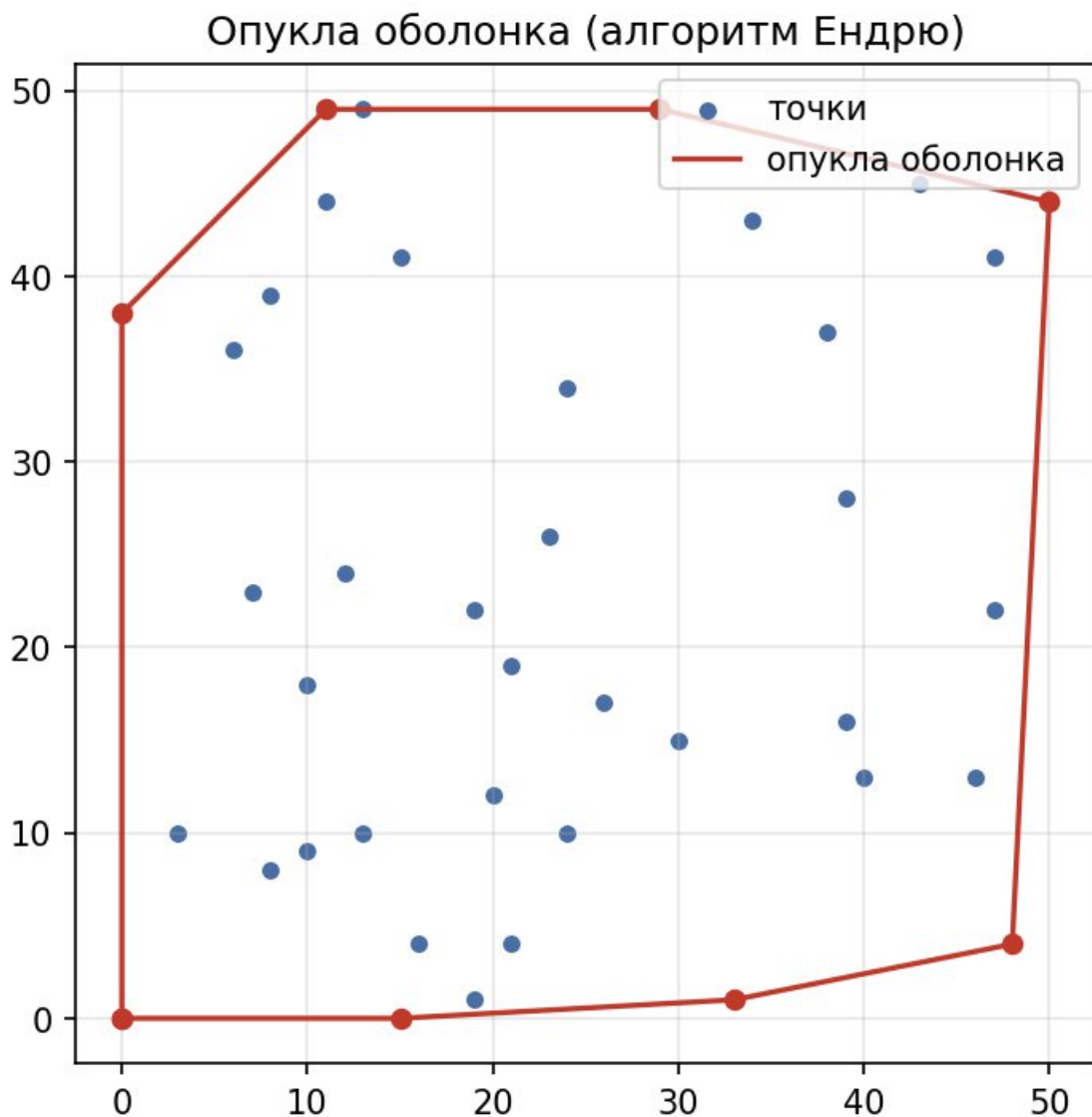
```
plt.gca().set_aspect("equal")
plt.title("Опукла оболонка (алгоритм Ендрю)")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

### Результат виконання програми:

Оболонка простого набору: [(0, 0), (4, 0), (4, 4), (0, 4)]  
 Випадкові точки: унікальних 40, вершин оболонки 8, площа 2242.5  
 Усі точки всередині оболонки: True; усі повороти ліві: True

| n      | вершин оболонки | час, с |
|--------|-----------------|--------|
| 1000   | 21              | 0.0013 |
| 10000  | 27              | 0.0119 |
| 100000 | 26              | 0.1787 |

### Графік порівняння:



*Висновок за результатами: для простого набору оболонку утворюють лише чотири кутові точки квадрата: точка (2, 0), що лежить на його стороні, виключається, оскільки умова  $\text{cross} \leq 0$  видаляє колінеарні точки, і оболонка містить тільки істотні вершини. Для випадкових даних усі точки перебувають усередині оболонки, а всі повороти — ліві, тобто оболонка опукла; кількість її вершин (8 із 40 точок) мала порівняно з n. Час побудови зростає*

*приблизно лінійно з невеликим множником  $\log n$  (при збільшенні  $n$  у 10 разів — приблизно у 7–13 разів), що відповідає оцінці  $O(n \log n)$  і не нагадує квадратичне зростання.*

---

## Контрольні питання

1. Що таке скалярний і векторний (косий) добуток? Який геометричний зміст кожного з них?
2. Як за знаком косого добутку визначити орієнтацію трьох точок? Чому цей прийом зручніший за тригонометрію?
3. Виведіть і поясніть формулу шнурівки для площі багатокутника. Що показує знак суми?
4. Сформулюйте умову перетину двох відрізків. Як обробляються колінеарні випадки?
5. Опишіть метод променя для визначення належності точки багатокутнику. Чому промінь через вершину потребує обережності?
6. Що таке опукла оболонка? Опишіть алгоритм Ендрю та його складність.
7. Чому для геометричних задач із цілими координатами обчислення можна робити точно, а з дійсними — потрібен допуск?
8. Які практичні задачі розв'язують алгоритми обчислювальної геометрії?

## Зміст звіту

1. Тема, мета роботи.
2. Ручні розрахунки та побудови (завдання 1, 2) і ручна побудова оболонки (завдання 4).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік опуклої оболонки.
5. Висновки.